



PROCESADOR MONOCICLO MIPS

Documentación Técnica — Rama: develop

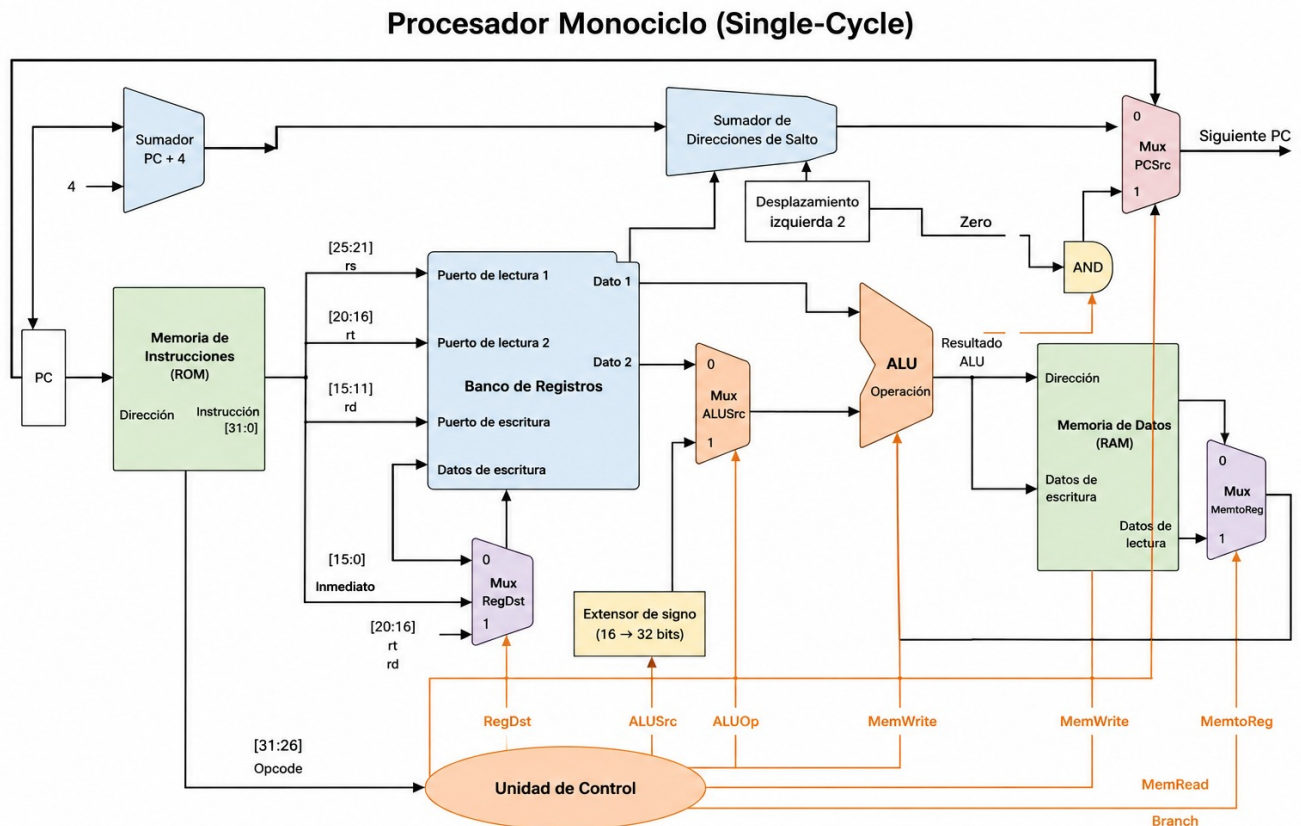
*Simulador de procesador monociclo 32 bits, arquitectura MIPS
con ensamblador integrado, interfaz gráfica y visualización del datapath.*

Andres Felipe Mogollon España

Juan Esteban Bedoya

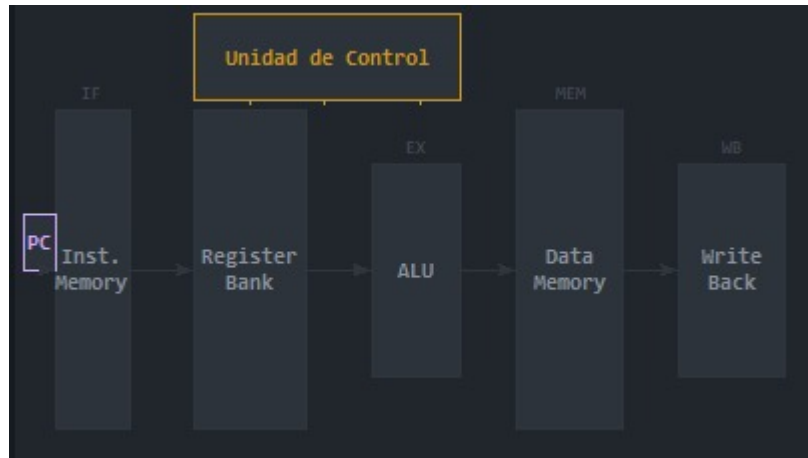
Michael Hurtado

Prof. David Andres Romero Arenas



IF	ID	EX	MEM	WB
Instruction Fetch	Instruction Decode	Execute	Memory Access	Write Back
Lee la instrucción de la memoria usando el PC	Decodifica los campos y lee registros	La ALU calcula el resultado u dirección	Lee o escribe en la memoria de datos	Escribe el resultado en el banco de registros

El hardware del monociclo está formado por bloques interconectados mediante señales de control y buses de datos:



Bloque	¿Qué hace?
PC (Program Counter)	Registro de 32 bits que guarda la dirección de la instrucción actual. Avanza solo (PC+4) o salta si hay branch/jump.
Memoria de instrucciones	ROM que almacena el programa en binario. Solo se puede leer, nunca escribir en tiempo de ejecución.
Banco de registros	32 registros de 32 bits. Permite leer dos registros al mismo tiempo y escribir uno. \$zero siempre vale 0.
Unidad de Control	Recibe el opcode y genera todas las señales de control (RegDst, ALUSrc, MemWrite, Branch, Jump...) que "dirigen el tráfico" por el datapath.
ALU	Ejecuta la operación aritmética o lógica (suma, resta, AND, OR...). Genera además la señal Zero usada por BEQ/BNE.
Memoria de datos	RAM de lectura/escritura. Solo se accede con LW (leer) o SW (escribir), controlada por MemRead y MemWrite.
Multiplexores (MUX)	Selectores de señal que eligen entre dos entradas según una señal de control. Por ejemplo, el MUX del operando B elige entre el valor del registro rt o el inmediato extendido.
Extensor de signo	Amplía el inmediato de 16 bits a 32 bits conservando el signo (bit 15 se replica hacia la izquierda).

0.3 Ejemplo Completo: Instrucción ADD

Veamos qué ocurre internamente cuando el procesador ejecuta la instrucción:

ADD \$t0, \$t1, \$t2

Significado: $\$t0 = \$t1 + \$t2$

Supongamos que antes de ejecutarla: $\$t1 = 7$ y $\$t2 = 5$. Al final esperamos: $\$t0 = 12$.

Memoria de Datos (palabras no-cero)

Dirección	Valor (hex)	Decimal
0x00000000	0x0000000c	12

Log de Ejecución

```
#000 PC=0x00000000 ADDI $t0, $zero, 7    ALU=0x000
#001 PC=0x00000004 ADDI $t1, $zero, 5    ALU=0x000
#002 PC=0x00000008 ADD $t0, $t1, $t2     ALU=0x000
#003 PC=0x0000000c SW $s0, 0($zero)      ALU=0x000
```

PROGRAMA TERMINADO
Total ciclos: 4 | PC final: 0x00000010

#	Etapas	Qué ocurre en el hardware
1	IF — Instruction Fetch	El PC vale, por ejemplo, 0x00000000. La Memoria de Instrucciones lee la palabra de 32 bits almacenada en esa dirección y entrega la instrucción codificada (0x012A4020 = ADD \$t0,\$t1,\$t2). Simultáneamente, el sumador de PC calcula $PC+4 = 0x00000004$ para la siguiente instrucción.
2	ID — Instruction Decode	El Decodificador de Instrucciones extrae los campos: opcode=000000, rs=01001 (\$t1=R9), rt=01010 (\$t2=R10), rd=01000 (\$t0=R8), funct=100000 (ADD). La Unidad de Control ve opcode=0 (R-type) y activa: RegDst=1, ALUSrc=0, RegWrite=1; desactiva MemRead, MemWrite, Branch y Jump. El Banco de Registros lee en paralelo \$t1 → 7 y \$t2 → 5.
3	EX — Execute	El MUX del operando B recibe ALUSrc=0, así que selecciona el valor del registro (\$t2=5) en lugar del inmediato. La Unidad de Control envía ALUControl=010 (ADD) a la ALU. La ALU suma: $7 + 5 = 12$. La señal Zero sale en 0 (resultado no es cero), así que no se tomará ningún branch.
4	MEM — Memory Access	La Unidad de Control tiene MemRead=0 y MemWrite=0, por lo que la Memoria de Datos permanece inactiva. No se lee ni se escribe nada en RAM. El valor que sale de esta etapa es simplemente el resultado de la ALU: 12.
5	WB — Write Back	El MUX de MemToReg recibe MemToReg=0, así que selecciona el resultado de la ALU (12) en lugar del dato de memoria. El MUX de RegDst recibe RegDst=1, así que el registro destino es rd (\$t0 = R8). El Banco de Registros escribe 12 en \$t0. ¡Instrucción completada! PC avanza a 0x00000004.

0.4 Flujo de Señales — Resumen Visual

El siguiente diagrama muestra cómo fluye la información y las señales de control para la instrucción ADD \$t0, \$t1, \$t2:

Registros	Memoria	Señales
Señales de Control activas		
☐ RegDst		Registro destino: 1=rd, 0=rt
☒ ALUSrc		Fuente B ALU: 1=Inmediato, 0=Reg
☐ MemToReg		WB fuente: 1=Memoria, 0=ALU
☒ RegWrite		Habilitar escritura en registros
☐ MemRead		Habilitar lectura de memoria
☐ MemWrite		Habilitar escritura en memoria
☐ Branch		Instrucción de salto condicional
☐ Jump		Salto incondicional J/JAL
☒ ALUOp		Código de operación ALU (2 bits)
☒ ALUControl		Control fino de ALU (3 bits)

Instrucción: ADD \$t0, \$t1, \$t2 (codificación: 0x012A4020)**Señales de control activas:**

RegDst=1 → Registro destino: rd (\$t0, R8)
 ALUSrc=0 → Operando B: desde registro (\$t2 = 5)
 ALUControl=010 → Operación: ADD
 MemRead=0, MemWrite=0 → Memoria inactiva
 MemToReg=0 → Dato al banco: resultado ALU (no memoria)
 RegWrite=1 → Escritura habilitada en banco de registros
 Branch=0, Jump=0 → PC ← PC+4 (secuencial)

Flujo de datos:

PC=0x00 → Mem.Instrucciones → Decoder → CU + Banco Reg
 \$t1=7, \$t2=5 → ALU (ADD) → resultado=12
 resultado=12 → MUX(MemToReg=0) → Banco Reg[\$t0] = 12 ✓

0.5 ¿Por qué "mono" ciclo?

La palabra "mono" (uno) se refiere a que el ciclo de reloj es lo suficientemente largo como para que cualquier instrucción—incluso la más lenta, como un LW que recorre todas las etapas incluyendo el acceso a memoria—pueda completarse. Esto tiene una consecuencia importante:

- Ventaja: diseño simple, sin necesidad de registros intermedios entre etapas ni lógica de control de hazards.
- Desventaja: el período del reloj lo fija la instrucción más lenta. Instrucciones rápidas (como AND) deben esperar igual que las lentas (como LW), desperdiciando tiempo.

Por eso, en la práctica real se usa el pipeline: divide el trabajo en etapas con registros intermedios, permitiendo que varias instrucciones se superpongan en el tiempo y reduciendo el período de reloj efectivo.

1. Descripción General

Este proyecto implementa un simulador completo de un procesador monociclo de 32 bits con arquitectura MIPS, desarrollado en Python. El simulador ejecuta instrucciones tipo R, I y J en un único ciclo de reloj por instrucción, siguiendo la pipeline clásica de cinco etapas:

IF → ID → EX → MEM → WB

La aplicación incluye un ensamblador MIPS de dos pasadas, una interfaz gráfica completa con Tkinter que visualiza el datapath de forma animada, y una CLI interactiva para entornos sin GUI.

1.1 Objetivos del Proyecto

- Implementar la arquitectura de un procesador monociclo MIPS de 32 bits.
- Simular el ciclo completo de ejecución instrucción por instrucción.
- Visualizar el flujo del datapath y las señales de control activas en cada etapa.
- Desarrollar un ensamblador MIPS de dos pasadas con soporte a etiquetas y pseudoinstrucciones.
- Aplicar principios SOLID y buenas prácticas de ingeniería de software en Python.

1.2 Tecnologías Utilizadas

Tecnología	Uso
Python 3.10+	Lenguaje principal — match/case, type hints, dataclasses
Tkinter	Interfaz gráfica nativa (incluida en Python)
pytest	Framework de testing unitario e integración
P00 + SOLID	Arquitectura y diseño del código
Git / GitHub	Control de versiones — flujo main/develop/feature/*

2. Arquitectura del Proyecto

El repositorio sigue una estructura modular con separación clara de responsabilidades. Cada capa tiene una responsabilidad única, facilitando el testing y la mantenibilidad:

```
unicycle_processor/
├── src/
│   ├── core/
│   │   ├── processor.py          # Orquestador del ciclo IF→ID→EX→MEM→WB
│   │   └── program_counter.py    # Registro PC – secuencial / branch / jump
│   ├── components/
│   │   ├── alu.py                # ALU 32 bits: ADD, SUB, AND, OR, SLT, NOR, XOR
│   │   ├── register_bank.py      # Banco de 32 registros × 32 bits
│   │   ├── instruction_decoder.py# Decodificador R/I/J + extensión de signo
│   │   ├── assembler.py         # Ensamblador MIPS de dos pasadas
│   │   └── mux.py                # Multiplexores del datapath
│   ├── memory/
│   │   └── memory.py             # InstructionMemory (ROM) + DataMemory (RAM)
│   ├── control/
│   │   └── control_unit.py       # Unidad de control: opcode → señales
│   └── ui/
│       └── app.py                # GUI completa con Tkinter
├── tests/
│   └── test_all.py               # 25+ tests unitarios e integración
├── programs/
│   └──                            # Programas .asm de ejemplo
└── main.py                      # Punto de entrada
```

2.1 Diagrama de Capas

El diseño sigue una arquitectura en capas donde cada módulo depende únicamente de los módulos de nivel inferior, sin referencias circulares:

Capa de Interfaz (UI)	src/ui/app.py	GUI Tkinter con datapath animado
Capa de Orquestación (Core)	src/core/processor.py	Coordina todas las etapas del ciclo
Capa de Componentes	src/components/*, src/control/*	ALU, Registros, Decoder, Ensamblador, Control
Capa de Memorias	src/memory/memory.py	InstructionMemory (ROM) y DataMemory (RAM)

3. Componentes del Sistema

3.1 MonocycleProcessor — Orquestador Principal

Archivo: `src/core/processor.py`

Clase central que coordina las cinco etapas del ciclo de instrucción en cada llamada a `step()`. Actúa como el "hilo conductor" del datapath y expone callbacks opcionales para que la GUI observe el estado en tiempo real.

Clase: MonocycleProcessor

- Instancia y conecta todos los componentes: ProgramCounter, RegisterBank, InstructionMemory, DataMemory, ALU, InstructionDecoder y ControlUnit.
- Método `step()` — ejecuta UN ciclo completo (IF → ID → EX → MEM → WB) y retorna un CycleSnapshot.
- Método `run(max_cycles)` — ejecuta el programa hasta HALT o hasta el límite de ciclos.
- Método `reset()` — reinicia PC, registros y memoria de datos sin borrar el programa cargado.
- Callback `on_cycle_complete` — permite que la GUI actualice la visualización tras cada ciclo.

Clase: CycleSnapshot

Captura inmutable del estado del procesador tras un ciclo. Contiene: número de ciclo, PC, instrucción raw, campos decodificados, señales de control, resultado de la ALU, dato de memoria, registro de escritura y snapshot completo de registros.

3.2 ProgramCounter — Contador de Programa

Archivo: `src/core/program_counter.py`

Registro de 32 bits que mantiene la dirección de la próxima instrucción. Soporta tres modos de actualización según el tipo de instrucción ejecutada:

Modo	Fórmula	Instrucciones
Secuencial	$PC \leftarrow PC + 4$	R-type, ADDI, LW, SW, BNE (no tomado)
Branch	$PC \leftarrow PC+4 + (\text{imm_signed} \ll 2)$	BEQ / BNE (cuando se toma el salto)
Jump	$PC \leftarrow \{PC+4[31:28], \text{addr26}, 00\}$	J, JAL

3.3 ALU — Unidad Aritmético-Lógica

Archivo: `src/components/alu.py`

ALU de 32 bits con signo. Recibe dos operandos y una señal ALUControl de 3 bits, y produce un resultado de 32 bits junto con señales de estado. Todos los valores se tratan internamente en complemento a dos.

Operación	ALUControl	Resultado	Instrucciones que lo usan
AND	000	$A \& B$	AND, ANDI
OR	001	$A \mid B$	OR, ORI
ADD	010	$A + B$	ADD, ADDI, LW, SW, LUI
SUB	110	$A - B$	SUB, BEQ, BNE
SLT	111	$(A < B) ? 1 : 0$	SLT, SLTI
NOR	100	$\sim(A \mid B)$	NOR
XOR	101	$A \wedge B$	XOR

Señales de estado generadas: Zero (usado por BEQ/BNE), Overflow (suma/resta con signo) y Negative (bit 31 del resultado).

3.4 RegisterBank — Banco de Registros

Archivo: `src/components/register_bank.py`

Banco de 32 registros de 32 bits estilo MIPS. La escritura está controlada por la señal RegWrite. El registro \$zero (R0) es de solo lectura y siempre vale 0.

- `read(reg)` — lectura combinacional; R0 siempre retorna 0.
- `write(reg, value, reg_write=True)` — escritura condicional; ignora escrituras a R0 silenciosamente.
- `read_dual(rs, rt)` — lee dos registros en paralelo (salidas del ID stage).
- `dump()` — retorna snapshot {nombre_registro: valor} para la GUI.
- `reset()` — limpia todos los registros sin tocar la memoria de instrucciones.

3.5 InstructionDecoder — Decodificador de Instrucciones

Archivo: `src/components/instruction_decoder.py`

Módulo combinacional que descompone los 32 bits de una instrucción en sus campos individuales y clasifica el tipo (R, I o J). También genera la extensión de signo del inmediato de 16 a 32 bits.

Formato de instrucciones MIPS

```
R-type: | opcode(6) | rs(5) | rt(5) | rd(5) | shamt(5) | funct(6) |
I-type: | opcode(6) | rs(5) | rt(5) | imm(16) |
J-type: | opcode(6) | address(26) |
```

La clasificación se realiza por opcode: opcode=0 → R-type, opcodes 0x02/0x03 → J-type, resto → I-type.

3.6 ControlUnit — Unidad de Control

Archivo: `src/control/control_unit.py`

Implementa la lógica combinacional que, dado el opcode (y funct para R-type), genera el vector completo de señales de control. Internamente utiliza una tabla de verdad precalculada para el mapping opcode → señales, y tablas secundarias para resolver ALUControl.

Señales de Control Generadas

Señal	Bits	Descripción
RegDst	1	0=rt es destino, 1=rd es destino (R-type)
ALUSrc	1	0=operando B desde registro, 1=inmediato extendido
MemToReg	1	0=resultado ALU al banco, 1=dato de memoria al banco
RegWrite	1	Habilita escritura en el banco de registros
MemRead	1	Habilita lectura de la memoria de datos (LW)
MemWrite	1	Habilita escritura en la memoria de datos (SW)
Branch	1	Instrucción de salto condicional (BEQ, BNE)
Jump	1	Salto incondicional (J, JAL)
ALUOp	2	Tipo de operación: 00=LW/SW, 01=BEQ, 10=R-type, 11=I-type ALU
ALUControl	3	Código final de operación enviado a la ALU

3.7 Memorias — InstructionMemory y DataMemory

Archivo: src/memory/memory.py

InstructionMemory (ROM)

- Capacidad: 1024 palabras de 32 bits (4 KB de instrucciones).
- Solo lectura. Las instrucciones se cargan mediante load_program().
- El acceso es word-addressed: dirección byte debe ser múltiplo de 4.
- Retorna NOP (0x00000000) para posiciones no cargadas.
- fetch(pc) — verifica alineación y rango antes de retornar la instrucción.

DataMemory (RAM)

- Capacidad: 2048 palabras de 32 bits (8 KB de datos).
- Lectura controlada por la señal MemRead; escritura por MemWrite.
- Interfaz byte-addressed, acceso word-aligned internamente.
- dump() — retorna solo posiciones no nulas, útil para la GUI.

3.8 Assembler — Ensamblador MIPS de Dos Pasadas

Archivo: src/components/assembler.py

Convierte código fuente MIPS en texto a una lista de enteros de 32 bits. Implementa el algoritmo clásico de dos pasadas:

- Primera pasada — escanea el código, registra todas las etiquetas con su dirección de instrucción (PC).
- Segunda pasada — genera el código binario, resolviendo referencias hacia adelante mediante la tabla de etiquetas.

Soporta comentarios con # o ;, líneas vacías, y cualquier combinación de mayúsculas/minúsculas en los mnemónicos. Los errores de ensamblado incluyen número de línea y texto original para facilitar la depuración.

Formatos de instrucción soportados

- R-type: mnem rd, rs, rt (ej: ADD \$t0, \$t1, \$t2)
- I-type aritmético: mnem rt, rs, imm (ej: ADDI \$t0, \$zero, 5)
- I-type memoria: mnem rt, imm(rs) (ej: LW \$t0, 4(\$sp))
- I-type branch: mnem rs, rt, label (ej: BEQ \$t0, \$t1, loop)
- J-type: mnem label (ej: J main)
- Pseudoinstrucciones: NOP, MOVE rd rs, LI rt imm, LA rt label

4. Conjunto de Instrucciones Soportadas

4.1 Instrucciones R-type (opcode = 0x00)

Mnemónico	Funct	Operación
ADD rd, rs, rt	0x20	$rd = rs + rt$
SUB rd, rs, rt	0x22	$rd = rs - rt$
AND rd, rs, rt	0x24	$rd = rs \& rt$
OR rd, rs, rt	0x25	$rd = rs rt$
XOR rd, rs, rt	0x26	$rd = rs \wedge rt$
NOR rd, rs, rt	0x27	$rd = \sim(rs rt)$
SLT rd, rs, rt	0x2A	$rd = (rs < rt) ? 1 : 0$

4.2 Instrucciones I-type

Mnemónico	Opcode	Operación
ADDI rt, rs, imm	0x08	$rt = rs + \text{SignExt}(imm)$
LW rt, imm(rs)	0x23	$rt = \text{MEM}[rs + imm]$
SW rt, imm(rs)	0x2B	$\text{MEM}[rs + imm] = rt$
BEQ rs, rt, off	0x04	if $rs == rt$: $PC = PC + 4 + (off \ll 2)$
BNE rs, rt, off	0x05	if $rs \neq rt$: $PC = PC + 4 + (off \ll 2)$
ORI rt, rs, imm	0x0D	$rt = rs \text{ZeroExt}(imm)$
ANDI rt, rs, imm	0x0C	$rt = rs \& \text{ZeroExt}(imm)$
SLTI rt, rs, imm	0x0A	$rt = (rs < imm) ? 1 : 0$
LUI rt, imm	0x0F	$rt = imm \ll 16$

4.3 Instrucciones J-type

Mnemónico	Opcode	Operación
J addr	0x02	$PC = \{PC + 4[31:28], \text{addr}, 00\}$
JAL addr	0x03	$\$ra = PC + 4; PC = \{PC + 4[31:28], \text{addr}, 00\}$

4.4 Pseudoinstrucciones

Pseudoinstrucción	Expansión equivalente
NOP	SLL \$zero, \$zero, 0 (instrucción 0x00000000)
MOVE rd, rs	ADD rd, \$zero, rs
LI rt, imm	ADDI rt, \$zero, imm
LA rt, label	ADDI rt, \$zero, dirección_de_label

5. Tabla de Señales de Control

La siguiente tabla muestra el vector de señales de control generado por la Unidad de Control para cada tipo de instrucción (X = don't care):

Instrucción	RegDst	ALUSrc	MemToReg	RegWrite	MemRead	MemWrite	Branch	Jump
R-type	1	0	0	1	0	0	0	0
ADDI	0	1	0	1	0	0	0	0
LW	0	1	1	1	1	0	0	0
SW	X	1	X	0	0	1	0	0
BEQ	X	0	X	0	0	0	1	0
BNE	X	0	X	0	0	0	1	0
J	X	X	X	0	0	0	0	1
JAL	X	X	X	1	0	0	0	1
ORI	0	1	0	1	0	0	0	0
ANDI	0	1	0	1	0	0	0	0
SLTI	0	1	0	1	0	0	0	0
LUI	0	1	0	1	0	0	0	0

6. Instalación y Ejecución

6.1 Prerrequisitos

- Python 3.10 o superior (se requiere match/case y type hints modernas).
- pip (incluido en Python 3.10+).
- Sistema operativo: Windows, macOS o Linux.
- Tkinter (incluido en la instalación estándar de Python; en algunas distros Linux puede requerir python3-tk).

6.2 Instalación

```
# 1. Clonar el repositorio (rama develop)
git clone -b develop https://github.com/FelipeMogollon1/unicycle_processor.git
cd unicycle_processor
```

```
# 2. (Opcional) Crear entorno virtual
python -m venv venv
source venv/bin/activate          # Linux / macOS
venv\Scripts\activate            # Windows
```

```
# 3. Instalar dependencias
pip install -r requirements.txt
```

6.3 Modos de Ejecución

Comando	Descripción
<code>python main.py</code>	Lanza la interfaz gráfica (modo principal)
<code>python main.py --demo</code>	Demo automática en consola — suma $1+2+3+4$
<code>python main.py --cli</code>	CLI interactiva — comandos <code>load/step/run/reset/regs/mem</code>
<code>python main.py --test</code>	Ejecuta la suite completa de tests con <code>pytest</code>
<code>pytest tests/ -v</code>	Ejecuta tests directamente con salida verbose

6.4 Comandos de la CLI Interactiva

- `load <archivo.asm>` — ensambla y carga un programa `.asm` en el procesador.
- `step` — ejecuta un ciclo y muestra el snapshot (PC, instrucción, señales, ALU, write-back).
- `run` — ejecuta el programa completo hasta `HALT`.
- `reset` — reinicia el procesador sin borrar el programa cargado.
- `regs` — muestra los registros con valor distinto de cero.
- `mem` — muestra las posiciones de memoria de datos no nulas.
- `quit / exit` — cierra la CLI.

7. Cobertura de Tests

El proyecto cuenta con 25+ tests organizados en el archivo `tests/test_all.py`. Los tests cubren tanto unidades individuales como la integración completa del procesador:

z	Tests	Casos cubiertos
TestALU	8	ADD, SUB, AND, OR, SLT, NOR, XOR, operando negativo, opcode inválido
TestRegisterBank	7	Inmutabilidad \$zero, R/W, máscara 32 bits, RegWrite=0, lectura dual, índice inválido, reset
TestControlUnit	6	R-type, LW, SW, BEQ, J, opcode inválido
TestDecoder	4	Instrucción R, I, J y extensión de signo
TestMemory	5	Fetch de instrucción, alineación, lectura/escritura de datos, señales MemRead/Write
TestProgramCounter	4	Secuencial, branch tomado, jump, reset
TestAssembler	5	NOP, ADDI, R-type, labels hacia adelante, pseudoinstrucciones
TestIntegration	6	Pipeline completo, bucle con BNE, LW/SW round-trip, BEQ, HALT, JAL

8. Programas de Ejemplo

El directorio `programs/` incluye 11 programas `.asm` que cubren un espectro progresivo de dificultad y sirven para demostrar y probar el procesador:

Archivo	Descripción
<code>01_aritmetica_basica.asm</code>	ADD, SUB, ADDI — operaciones aritméticas básicas
<code>02_operaciones_logicas.asm</code>	AND, OR, XOR, NOR, ANDI, ORI — lógica bit a bit
<code>03_load_store.asm</code>	LW, SW — acceso a memoria de datos
<code>04_condicionales.asm</code>	BEQ, BNE — saltos condicionales
<code>05_bucle_suma.asm</code>	Bucle con BNE — suma acumulativa
<code>06_multiplicacion.asm</code>	Multiplicación por sumas sucesivas
<code>07_fibonacci.asm</code>	Secuencia de Fibonacci con bucle y memoria
<code>08_maximo_arreglo.asm</code>	Búsqueda del máximo en un arreglo
<code>09_mcd_euclides.asm</code>	Máximo común divisor — algoritmo de Euclides
<code>10_bubble_sort.asm</code>	Ordenamiento burbuja sobre un arreglo en memoria
<code>example.asm</code>	Ejemplo de la demo consola — suma $1+2+3+4$

9. Flujo Git y Convenciones

9.1 Estructura de Ramas

- `main` — código estable, releases finales.
- `develop` — integración de features (rama de esta documentación).
- `feature/<nombre>` — desarrollo de funcionalidades individuales (ej: `feature/alu`, `feature/gui`).
- `bugfix/<nombre>` — corrección de errores (ej: `bugfix/pc-jump`).

9.2 Conventional Commits

El proyecto sigue el estándar Conventional Commits para los mensajes de commit:

```
feat(alu): add XOR operation support
fix(decoder): correct sign extension for 16-bit immediate
docs(readme): add instruction set table
test(processor): add BEQ integration test
refactor(memory): extract base class for ROM and RAM
```


10. Características de la Interfaz Gráfica

La GUI (`src/ui/app.py`) está construida con Tkinter y ofrece las siguientes funcionalidades:

- Visualización animada del datapath — cada etapa (IF, ID, EX, MEM, WB) se ilumina durante su ejecución.
- Editor de código ensamblador integrado con resaltado de sintaxis.
- Indicadores de señales de control con LEDs de color (verde/rojo según estado).
- Panel de registros — muestra en tiempo real el valor de los 32 registros MIPS.
- Panel de memoria de datos — visualiza las posiciones no nulas de la DataMemory.
- Log de ciclos — historial de mnemónicos, valores ALU y registros modificados por ciclo.
- Modo automático con velocidad ajustable (0.05 – 2.0 segundos por ciclo).
- Ejecución paso a paso (Step) o continua (Run).
- Botón Reset para reiniciar sin perder el programa cargado.